Отчет по лабораторной работе №9

Дисциплина: Архитектура ЭВМ

Тема: Понятие подпрограммы. Отладчик GDB.

Студент: А н ь с я о я н ь

Группа: НПИ-01-25

1. Цель работы

Приобретение навыков написания программ с использованием подпрограмм. Знакомство с методами отладки при помощи GDB и его основными возможностями.

2. Выполнение работы

2.1. Реализация подпрограмм в NASM

Я создал файл `lab09-1.asm` и реализовал программу вычисления арифметического выражения f(x)=2x+7 с использованием подпрограммы `_calcul`.

Команды: nasm -f elf32 lab09-1.asm

Команды: i686-linux-gnu-ld -o lab09-1 lab09-1.o

Команды: qemu-i386 ./lab09-1

Ожидаемый результат: Введите x: 5

2x+7=17

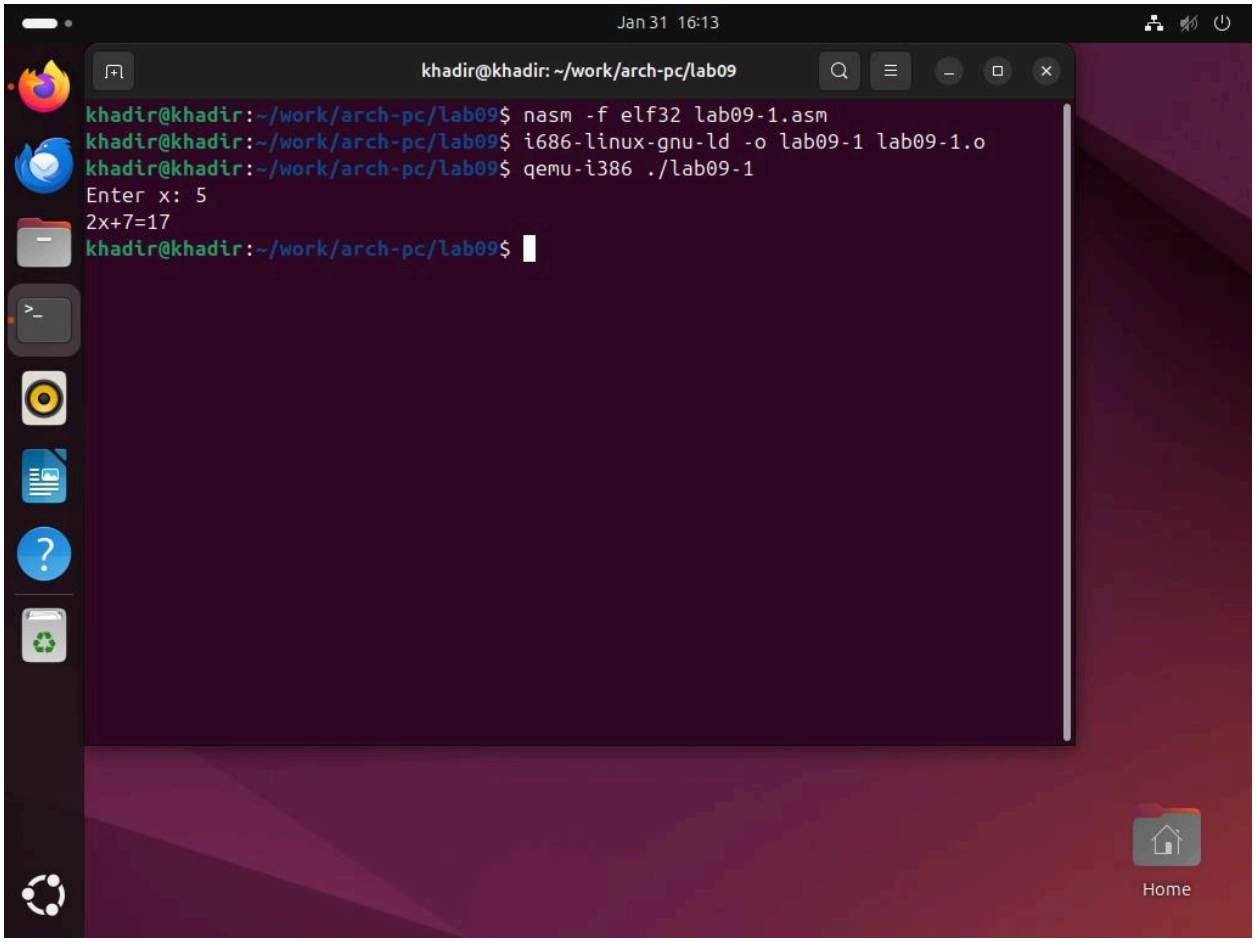

Рис. 9.1. Вычисление 2x+7 в программе lab09-1.asm

2.2. Вычисление сложной функции f(g(x))

Я изменил программу, добавив подпрограмму `_subcalcul` для вычисления g(x)=3x−1. Теперь программа вычисляет f(g(x))=2(3x−1)+7.

Команды: nasm -f elf32 lab09-1.asm

Команды: i686-linux-gnu-ld -o lab09-1 lab09-1.o

Команды: qemu-i386 ./lab09-1

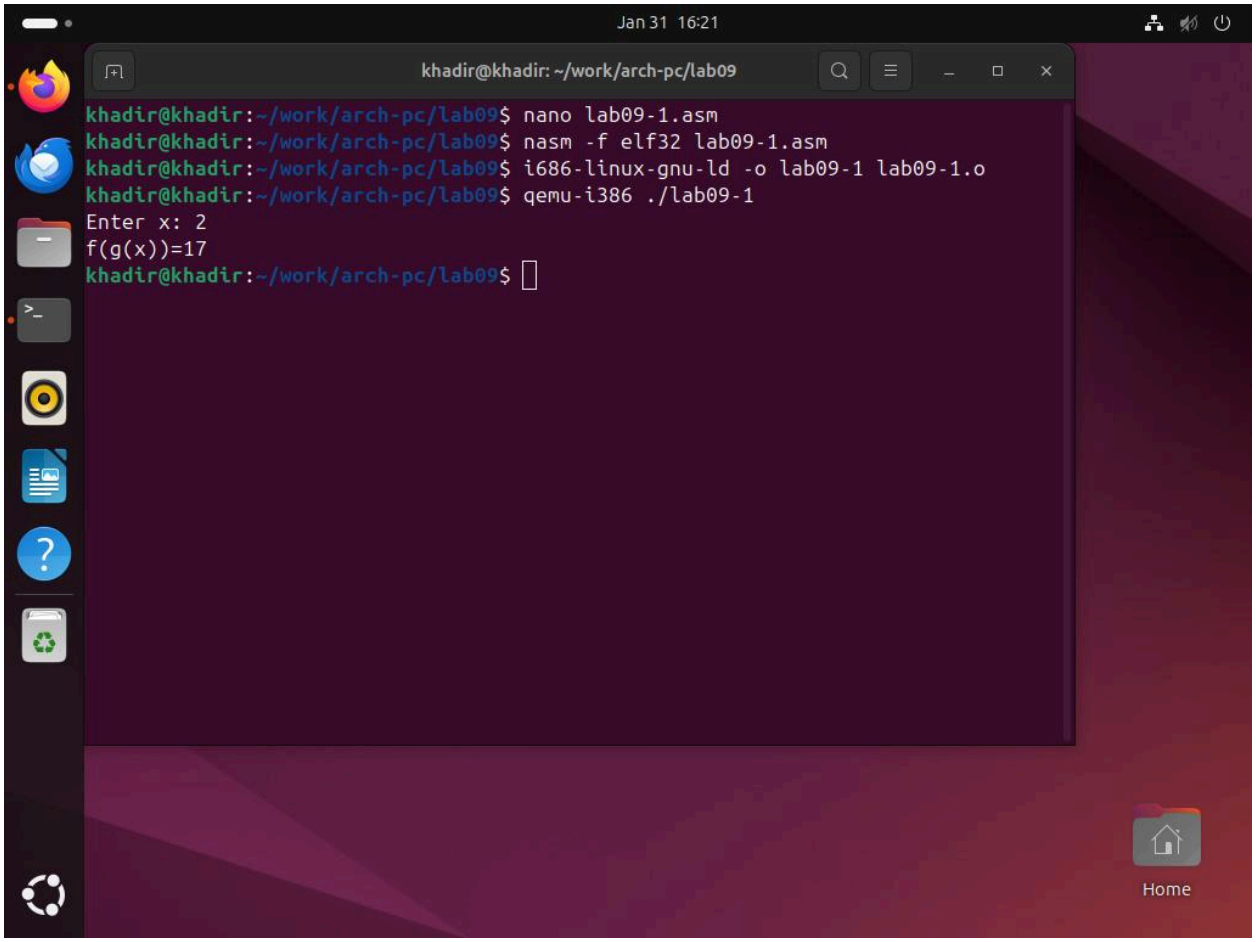

Рис. 9.2. Работа подпрограмм при вычислении f(g(x)).

2.3. Работа с отладчиком GDB

Я от транслировал программу `lab09-2.asm` с ключом `-g` для сохранения отладочной информации и запустил её в GDB.

Команды: : nasm -f elf32 -g lab09-2.asm

Команды: i686-linux-gnu-ld -o lab09-2 lab09-2.o

Команды: `i686-linux-gnu-objdump`

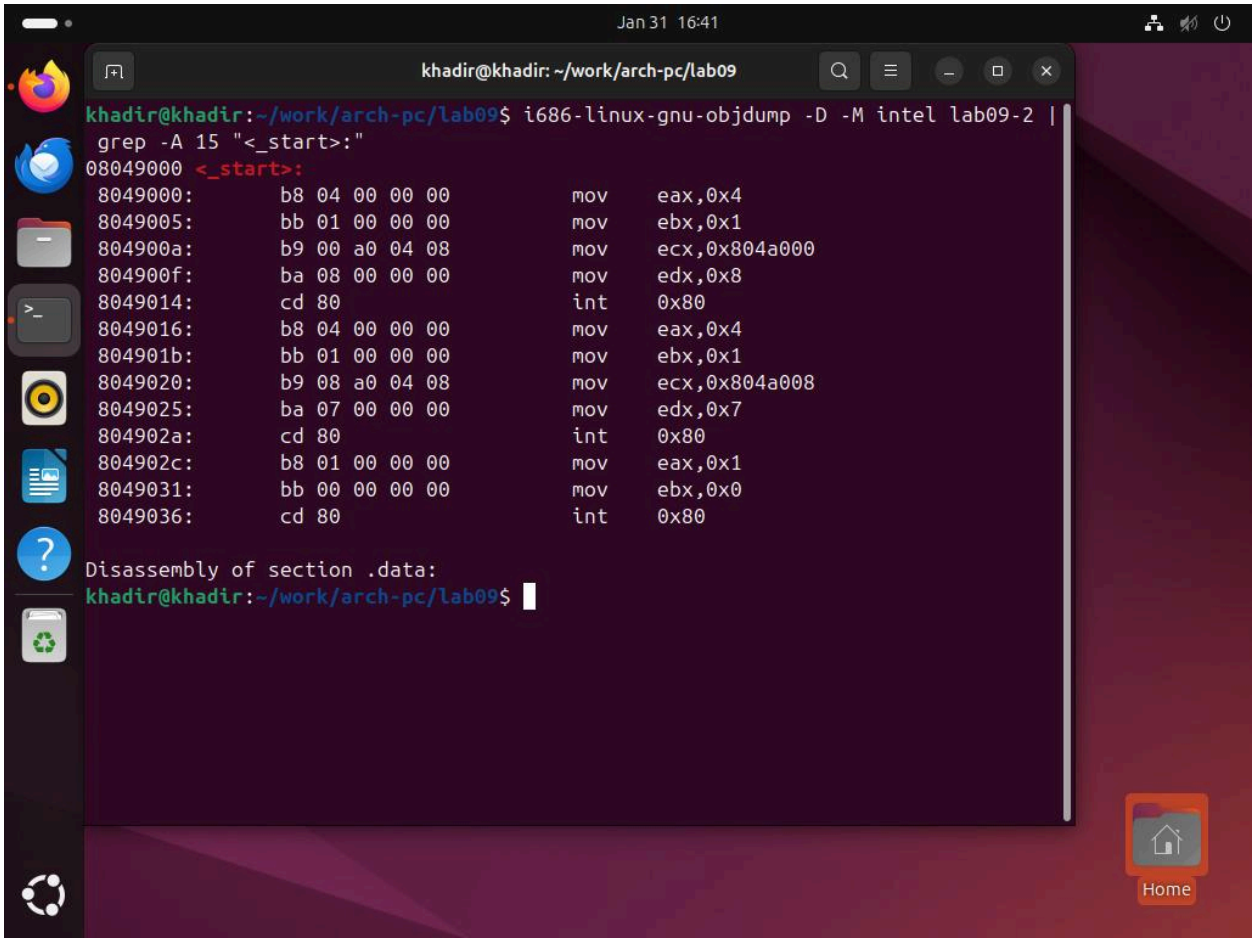

Рис. 9.3. Запуск программы lab09-2 в GDB и просмотр кода командой disassemble _start.

Для более удобного анализа я использовал режим псевдографики.

Команды: set disassembly-flavor intel

Команды: layout asm

Команды: `qemu-i386 -d cpu lab09-2 | head -n 20`

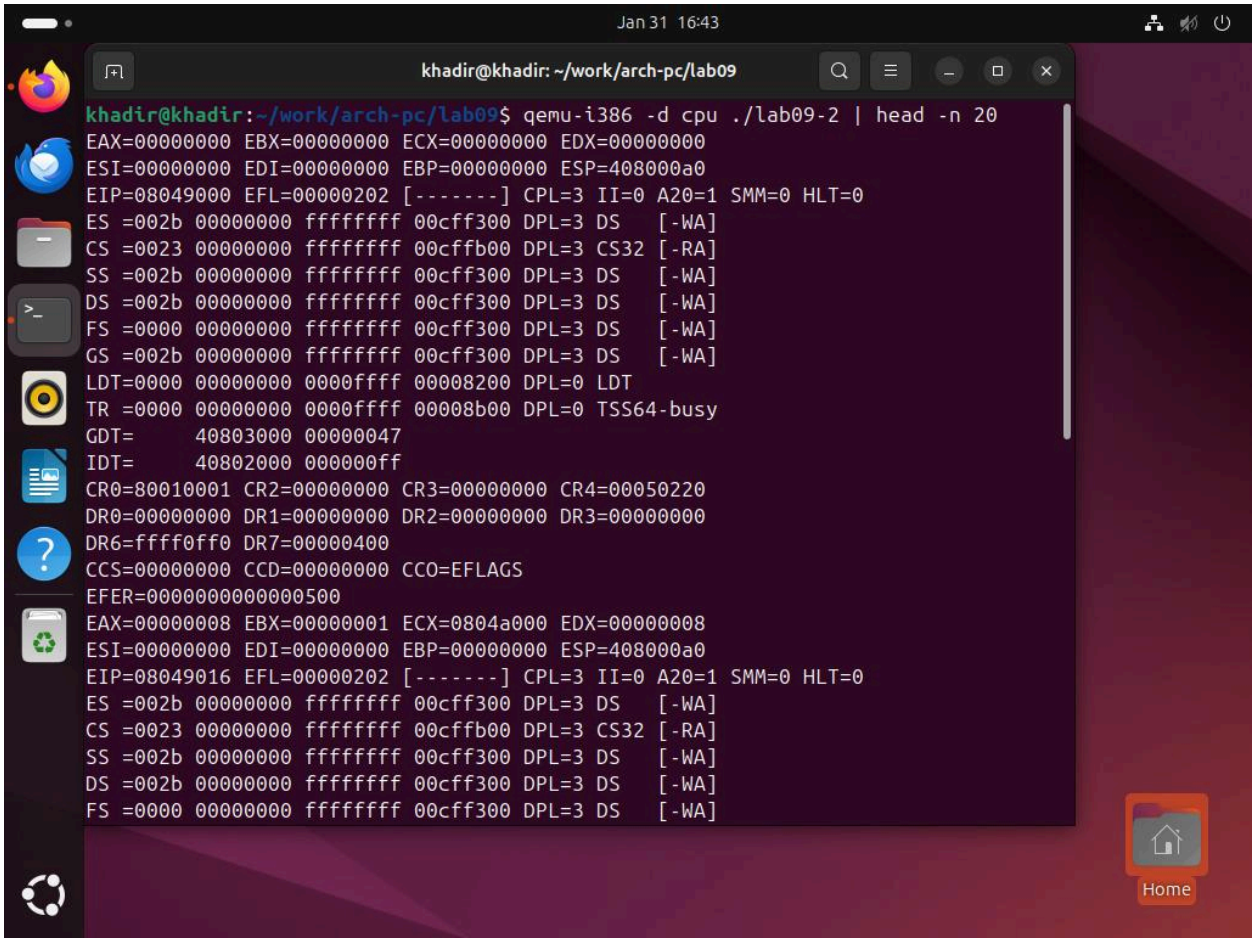

Рис. 9.4. Работа в режиме псевдографики GDB (отображение регистров и кода).

2.4. Анализ стека и аргументов командной строки

Я исследовал расположение аргументов командной строки в стеке с помощью отладчика.

Команды: gdb –args lab09-3 аргумент1 аргумент2 ’аргумент 3’

Команды: break_start

Команды: run

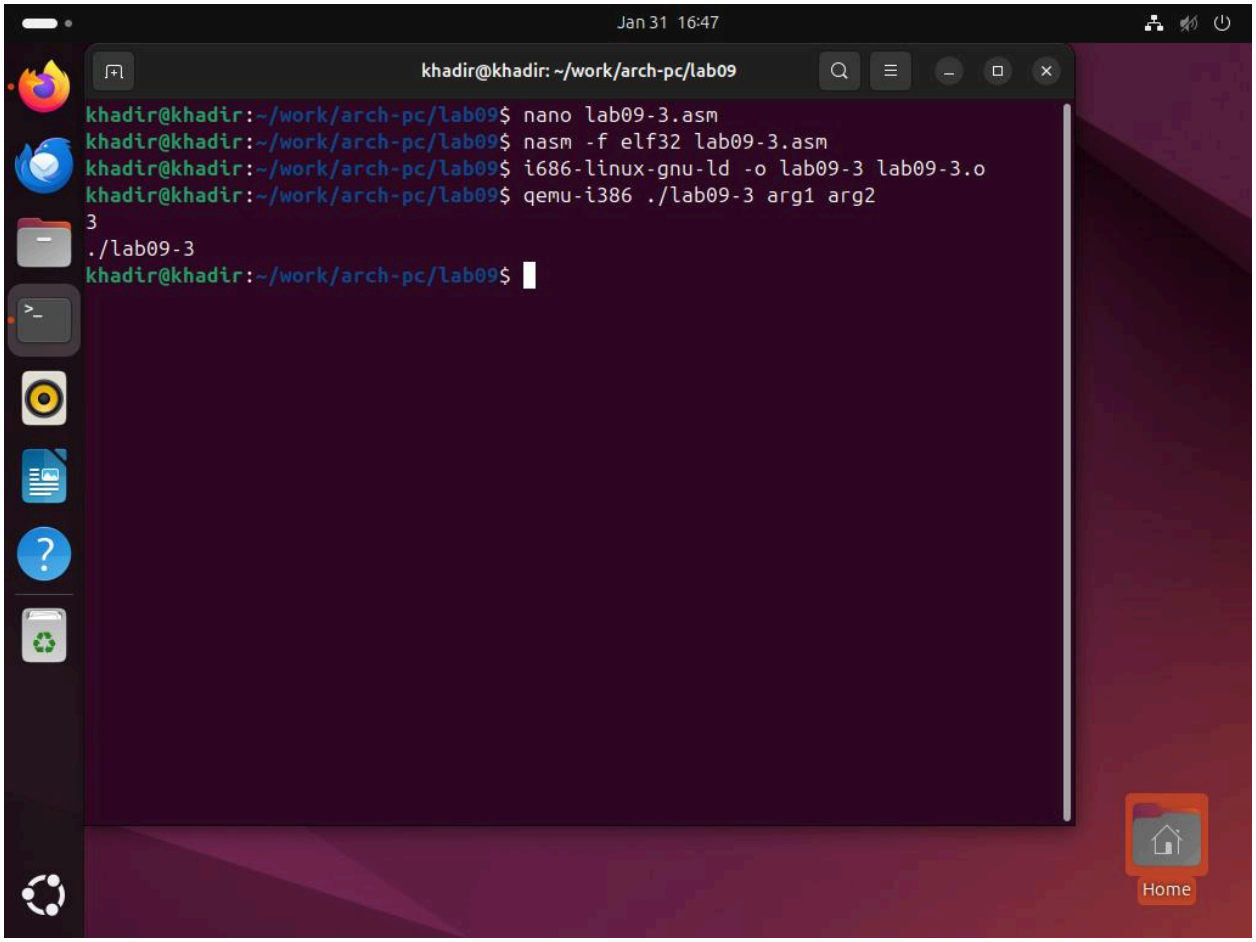

Рис. 9.5. Исследование стека в GDB: проверка количества аргументов и их адресов.

3. Самостоятельная работа

Задание 1: Подпрограмма (Вариант 12)

Я переписал программу для вычисления функции f(x)=15x−9, оформив расчет в виде подпрограммы.

Команды: nasm -f elf32 lab09-4.asm

Команды: i686-linux-gnu-ld -o lab09-4 lab09-4.o

Команды: qemu-i386 ./lab09-4

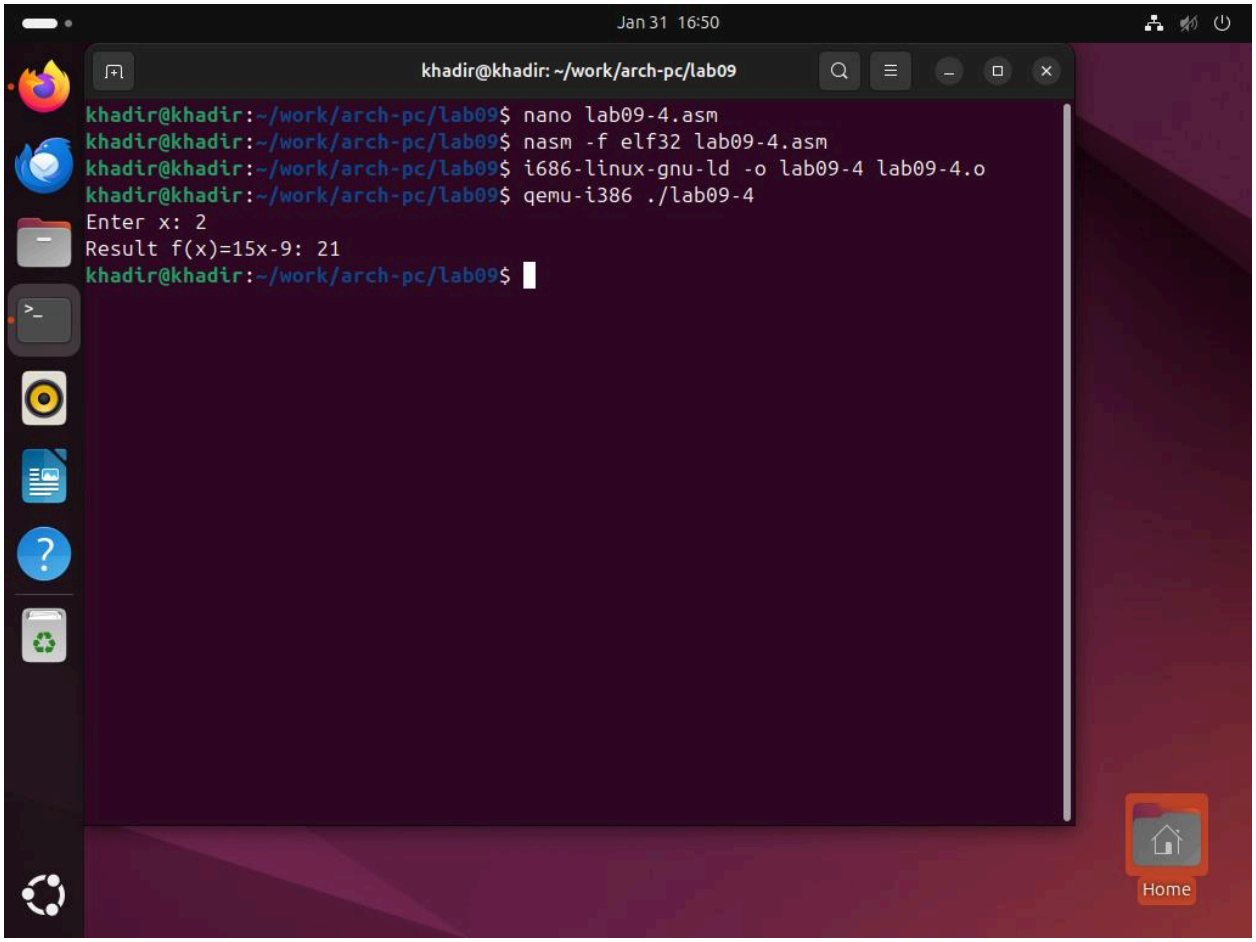

Рис. 9.6. Работа программы с использованием подпрограммы для варианта 12.

Задание 2: Исправление ошибки в GDB

Я проанализировал программу `lab09-5.asm`, которая должна вычислять (3+2)∗4+5=25. С помощью GDB было обнаружено, что ошибка заключалась в порядке использования регистров.

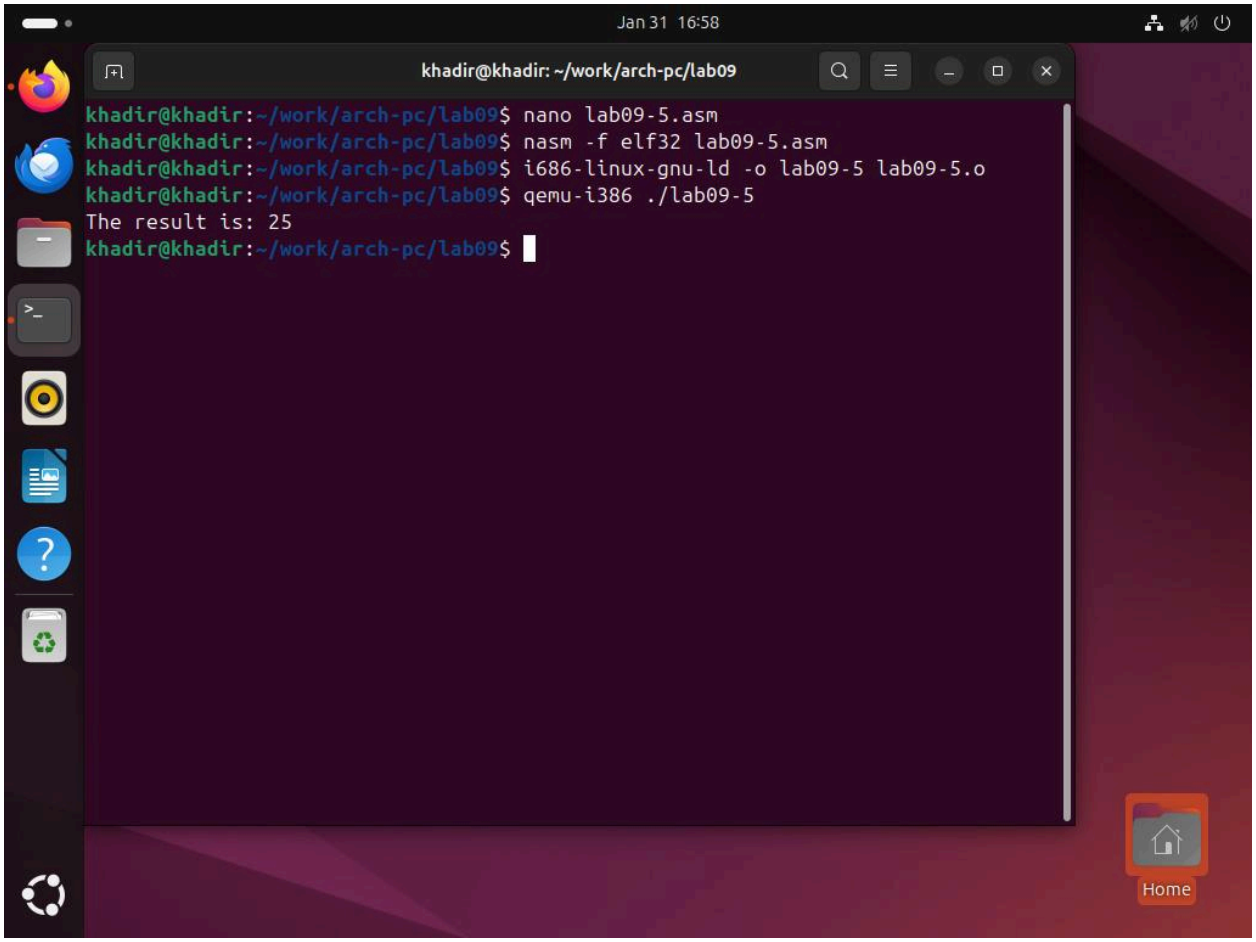

Рис. 9.7. Поиск ошибки в регистрах eax/ebx через отладчик.

4. Контрольные вопросы

1. Какие средства используются для подпрограмм? Инструкция `call` для вызова и `ret` для возврата.
2. Механизм вызова? `call` сохраняет адрес следующей команды в стек и переходит к подпрограмме. `ret` извлекает этот адрес и возвращается.
3. Использование стека? Стек хранит адрес возврата. Также через стек можно передавать параметры.
4. Назначение операнда в ret? Числовой операнд в `ret N` позволяет очистить `N` байт из стека после возврата (обычно параметры).
5. Для чего нужен отладчик? Для поиска ошибок, контроля регистров и пошагового выполнения кода.
6. Отладочная информация? Это данные о соответствии машинного кода строкам исходника. Нужен флаг `-g` при компиляции.
7. Термины:
    - Breakpoint: точка остановки на строке/адресе.
    - Watchpoint: остановка при изменении значения переменной.

    - Call stack: список активных подпрограмм.
8. Команды GDB: `run` (запуск), `break` (точка останова), `info registers` (просмотр регистров), `stepi` (шаг), `continue`(продолжение).

5. Выводы

В ходе работы я изучил механизм работы подпрограмм и команд `call` и `ret`. Также я освоил базовые команды отладчика GDB, которые позволяют контролировать состояние регистров и памяти в процессе выполнения программы.